Informe de seguridad del frontend

PharmaSoft · Actividad autónoma / Reto 02 · Lenguaje de Programación II

BORRADOR: faltan los resultados y capturas reales de las 24 pruebas.

C1. Resumen de casos

La revisión cubre login, sesión, interceptores, guards, menú por permisos y aviso de expiración. Se prepararon **20 casos** de la Parte A (16 obligatorios y 4 propios de grupos diferentes) y los **4 casos** B-01 a B-04: **24 pruebas en total**. Cada una necesita resultado observado y evidencia de F12 o de la interfaz.

Estado inicial: no se han ejecutado las 24 pruebas en la computadora del estudiante. Cero casos aprobados o fallidos significa que aún no se han medido, no que la aplicación esté libre de errores.

Grupo	Casos	Pasan	Fallan	Pendientes
Login	5	0	0	5
Sesión	5	0	0	5
Interceptor	4	0	0	4
Guards	3	0	0	3
Menú	2	0	0	2
401 / 403	1	0	0	1
Aviso	4	0	0	4
TOTAL	**24**	**0**	**0**	**24**

C2. Fichas de hallazgos

Aún no hay hallazgos confirmados. Por cada prueba que falle, incorporar una ficha con: **ID, descripción, severidad justificada, captura, causa probable (archivo y método), corrección propuesta y verificación posterior**.

Caso S-04: la independencia de **sessionStorage por pestaña** puede dejar otra pestaña iniciada después de salir en la primera. Registrar el resultado observado y evaluar una sincronización mediante BroadcastChannel.

C3. Preguntas de análisis

1. ¿Por qué el backend debe seguir validando permisos si el frontend oculta acciones?

El frontend está bajo control del navegador del usuario: puede modificarse el DOM, enviarse una solicitud manual o saltarse un guard. Por eso **permisoGuard** y ***appSiPermiso** solo mejoran la navegación; no constituyen una defensa suficiente. Cada endpoint debe comprobar el JWT y los permisos en el servidor antes de consultar o cambiar datos. La prueba E-01 está diseñada para intentar una operación restringida con VENDEDOR sobre datos de prueba: el resultado que se debe verificar es HTTP 403 sin cambiar el recurso y conservando la sesión. La conclusión definitiva dependerá de esa evidencia real.

2. ¿Qué ventajas y riesgos presentan sessionStorage, localStorage y cookies HttpOnly?

sessionStorage mantiene la sesión en la pestaña y generalmente se descarta cuando termina; su inconveniente es que un script malicioso en la misma página (XSS) puede leer el token. **localStorage** es igualmente accesible desde JavaScript y persiste normalmente después de cerrar el navegador, lo que puede ampliar la exposición. Una cookie **HttpOnly, Secure y SameSite** evita que el token sea leído por JavaScript, pero requiere una configuración adecuada contra CSRF y soporte del backend. Para una botica real elegiría la cookie HttpOnly con controles de servidor, CSP, autenticación sólida y vencimiento apropiado; en este laboratorio se mantiene sessionStorage por el contrato de la sesión 10.

Análisis y verificación final

3. ¿Por qué cerrar sesión por 401 pero no por 403?

Un HTTP **401** señala autenticación ausente, vencida o inválida: la SPA debe borrar la sesión y volver al login. El código de **core/interceptors/error-interceptor.ts** lo hace para 401 que no viene del login. Un HTTP **403** significa que la identidad puede ser válida pero carece de permiso: se informa el acceso denegado sin cerrar sesión. Si se expulsara al usuario por 403, un vendedor perdería el trabajo al abrir accidentalmente una operación restringida; si se conservara una sesión inválida por 401, las siguientes solicitudes fallarían repetidamente.

4. ¿Qué es una redirección abierta y cómo se evita con returnUrl?

Se produce cuando una aplicación acepta sin control una URL proporcionada por terceros y envía allí al usuario, incluso a sitios falsos. En L-04 se introduce **returnUrl=https://sitio-falso.com**. El método **destino()** de **features/auth/pages/login/login.ts** solo admite rutas que empiezan por una barra **/** pero no por dos **//**; en otro caso devuelve **/inicio**. La prueba debe verificar que el navegador permanece dentro de PharmaSoft y nunca visita el dominio externo.

Funcionalidades implementadas

El aviso utiliza **segundosRestantes** y **porVencer**, muestra mm:ss y permite cerrar sesión u ocultar la alerta. Se limpian setInterval y setTimeout al salir. La ruta **/mi-cuenta** utiliza authGuard y consulta **GET /api/v1/auth/me** con los interceptores ya existentes. Compara permisos y roles del servidor con la sesión guardada y muestra un aviso si difieren.

Pendiente para la entrega

Ejecutar los 24 casos, ocultar JWT completos en todas las capturas, actualizar los totales de C1, completar una ficha C2 por fallo, restaurar app.jwt.expiracion-minutos a 60, regenerar el PDF final y crear los commits reales en días distintos antes del Pull Request.